.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 2
Programación dinámica

8 de octubre de 2023

Geronimo Palavecino
109681

Macarena Vita Sanchez
109377

Mariana Galdo Martinez
105658

1. Introducción

En este informe, abordamos el desafío que enfrenta Scaloni después de analizar a sus rivales: la planificación del entrenamiento de la selección nacional. Scaloni ha establecido un cronograma de entrenamientos para los próximos n días, donde la ganancia de cada día de entrenamiento es proporcional al esfuerzo invertido. Sin embargo, el agotamiento de los jugadores es un problema, y Scaloni debe decidir cuándo permitirles descansar para restaurar su energía. El objetivo principal es maximizar la ganancia total acumulada durante el período de entrenamiento. Menotti, experto en el fútbol argentino, ha recomendado la Programación Dinámica como una solución a este desafío.

Resolución

2. Análisis del Problema

El problema en cuestión se enfoca en la optimización de la secuencia en la que Scaloni debe programar los entrenamientos para maximizar el rendimiento de sus jugadores. Para abordar este problema, identificamos cuatro elementos clave que influyen en la toma de decisiones:

- **Número de Entrenamientos (n):** Representa la cantidad de días para los cuales Scaloni debe definir un cronograma de entrenamiento.
- **Número de Día de Entrenamiento (i):** Indica la cantidad de días transcurridos desde el primer entrenamiento en el cronograma.
- **Cantidad de días post descanso (j):** Corresponde a la cantidad de días transcurridos desde el último descanso.
- **Esfuerzo Diario (e_i):** Este valor refleja el nivel de esfuerzo previsto para cada día de entrenamiento.
- **Secuencia de Desgaste de Energía (s_j):** Representa la secuencia de desgaste energético de los jugadores. Después de un día de descanso, esta secuencia se reinicia.

Una representación gráfica del problema analizado es:



días	1	2	3	4	5
esfuerzo	90	72	54	28	39
energía	85	70	1	44	30

Figura 1: Contexto del problema

3. Análisis de la Solución

A continuación, presentamos un análisis detallado de nuestra solución, desglosado paso a paso:

- **Para** 0 **Días de Entrenamiento y** 0 **Días Después del Último Descanso:** En este caso, la ganancia es 0, ya que no ha habido entrenamientos ni desgaste de energía.
- **Para** 1 **Día de Entrenamiento y** 1 **Día Después del Último Descanso:** En esta situación, la ganancia será el mínimo entre el esfuerzo diario (e_1) y la cantidad de energía disponible (s_1).
- **Para** 2 **Días de Entrenamiento:** La ganancia máxima se alcanzará al seleccionar la mejor opción entre descansar en este día, descansar el día anterior o realizar el entrenamiento en el día actual y el anterior.

Además, es importante destacar que no se observa un aumento en la ganancia al programar días de descanso consecutivos superiores a 1. Esto se debe a que la recuperación de energía se logra en un solo día.

Basándonos en este análisis inicial, hemos formulado la ecuación de recurrencia correspondiente:

Para $j \geq 1$:

$$\text{OPT}(e_i, s_j) = (\text{mín}\{e_i, s_j\} + \text{OPT}(e_{i-1}, s_{j-1}))$$

Para $j = 0$:

$$\text{OPT}(e_i, s_j) = \text{máx}\{\text{OPT}(e_{i-1}, s_k)\},\ 1 \leq k \leq i-1$$

En el caso de $j = 0$, como ya sabemos que no es conveniente descansar dos días seguidos, podemos buscar el máximo empezando desde $k = 1$.

A partir de esta ecuación de recurrencia decidimos simular el comportamiento del programa en una planilla de cálculo para poder mejorar la comprensión del problema. Armamos una tabla con i en el eje horizontal y j en el eje vertical.

Consideremos los siguientes valores para e_i y s_j en un plazo de 10 días.

e_i	0	75	77	54	36	78	36	44	77	48	65
s_j	0	74	69	65	65	64	37	23	22	22	7

Omitiendo los primeros dos casos que tienen ganancia 0 para $i = j = 0$ y para $i = 1$ y $j = 0$, si en el primer día de entrenamiento se decide entrenar, la ganancia será el valor mínimo entre el esfuerzo e_1 y la energía disponible s_1. En este caso, s_1 es menor. Por lo tanto, la ganancia acumulada es 74. Para el caso de $i = 1$ y $j = 2$, no tiene sentido considerarlo, ya que, no es posible que hayan pasado 2 días desde el descanso si estamos en el día 1 de entrenamiento. Por esta razón, no completamos los valores de $j > 1$ para $i = 1$ en nuestra planilla.

Tabla hasta $i = 1$ y $j = 1$

j												
10												
9												
8												
7												
6												
5												
4												
3												
2												
1		74										
0	0	0										
	0	1	2	3	4	5	6	7	8	9	10	i

Figura 2: Seguimiento hasta $i = 1$ y $j = 1$

En este punto, nos damos cuenta que, en nuestra tabla, cada valor de i le van a corresponder i+1 valores en j, excepto para $i = 0$. Es decir, por cada día i de entrenamiento, van a haber $i + 1$ posibles ganancias distintas a obtener, según lo que haya pasado en los días anteriores.

Si seguimos para $i = 2$, tendremos que ver las combinaciones con $j = 0$, $j = 1$ y $j = 2$. Para el primer caso, estaríamos en el caso particular donde ese día se descansa. Considerando que ya sabemos que descansar dos días seguidos no es ventajoso, la ganancia posible en esa situación es la ganancia en $i = 1$ y $j = 1$, que es 74. Para $j = 2$, al comparar 77 con 69, obtenemos que la ganancia de ese día es 69. Pero, como ya es el segundo día desde el último descanso, el día anterior también se entrenó, por lo que tenemos que sumar la ganancia del día anterior para $i = 1$ y $j = 1$. Entonces, la ganancia acumulada para $j = 2$ es 143.

Tabla hasta $i = 2$ y $j = 2$

j												
10												
9												
8												
7												
6												
5												
4												
3												
2			143									
1		74	74									
0	0	0	74									
	0	1	2	3	4	5	6	7	8	9	10	i

Figura 3: Seguimiento hasta $i = 2$ y $j = 2$

Para el día 3, hacemos lo mismo que en el día 2, pero completamos hasta $j \leq 3$. En el caso de $j = 0$, vamos a tener más valores a comparar para encontrar la máxima ganancia en el día anterior. Es decir, cada vez que buscamos el máximo cuando se descansa en algún día i, vamos a tener que comparar $i - 1$ ganancias posibles en el día anterior. Por eso, para $i = 3$, nos tenemos que fijar si para $i = 2$, es mayor la ganancia en $j = 1$ o $j = 2$.

Tabla hasta $i = 3$ y $j = 3$

j												
10												
9												
8												
7												
6												
5												
4												
3				197								
2			143	128								
1		74	74	128								
0	0	0	74	143								
	0	1	2	3	4	5	6	7	8	9	10	i

Figura 4: Seguimiento hasta $i = 3$ y $j = 3$

Si seguimos los mismos pasos para el resto de los días, nuestra tabla estará completa con 66 valores. En este punto, para determinar cuál es la ganancia máxima posible, solo tenemos que encontrar el máximo valor en la columna 10, es decir, en el último día de entrenamiento. En nuestro ejemplo, la ganancia máxima posible es 523.

Tabla completa

j												
10											407	
9										400	369	
8									378	347	391	
7								356	325	369	452	
6							333	302	346	429	501	
5						297	265	309	392	464	490	
4					233	229	265	328	416	426	523	
3				197	164	229	284	351	378	458	520	
2			143	128	164	248	307	313	410	455	469	
1		74	74	128	179	271	269	341	407	404	481	
0	0	0	74	143	197	233	297	333	356	416	464	
	0	1	2	3	4	5	6	7	8	9	10	i

Figura 5: Tabla completa con la máxima ganancia

Una vez que ya encontramos la máxima ganancia, podemos reconstruir el camino que hicimos para llegar a este valor. Dado que calculamos las ganancias para cada día i con $j > 0$, teniendo en consideración las ganancias del día anterior en $i - 1$ y $j - 1$, estos valores se convierten en componentes de la solución. Es decir, empezando en $i = 10$, $j = 4$, el siguiente valor que va a formar parte de la solución se encuentra en $i = 9$, $j = 3$. Seguimos de esta manera hasta que $j = 0$.

Reconstrucción del camino

j												
10											407	
9										400	369	
8									378	347	391	
7								356	325	369	452	
6							333	302	346	429	501	
5						297	265	309	392	464	490	
4					233	229	265	328	416	426	523	
3				197	164	229	284	351	378	458	520	
2			143	128	164	248	307	313	410	455	469	
1		74	74	128	179	271	269	341	407	404	481	
0	0	0	74	143	197	233	297	333	356	416	464	
	0	1	2	3	4	5	6	7	8	9	10	i

Figura 6: Reconstrucción del camino hasta encontrarnos con $j = 0$

Cuando $j = 0$, como nos indica que ese día se descansa, para seguir con el camino, solo tenemos que buscar el máximo valor en el día anterior. Ese valor tiene que coincidir con la ganancia obtenida en $j = 0$, ya que, al ser día de descanso, la ganancia no debería aumentar. Luego de este punto, podemos reanudar con el mismo procedimiento que seguimos antes. Una vez que llegamos a $i = 0$, $j = 0$, ya tenemos nuestro camino completo.

Reconstrucción del camino

j												
10											407	
9										400	369	
8									378	347	391	
7								356	325	369	452	
6							333	302	346	429	501	
5						297	265	309	392	464	490	
4					233	229	265	328	416	426	523	
3				197	164	229	284	351	378	458	520	
2			143	128	164	248	307	313	410	455	469	
1		74	74	128	179	271	269	341	407	404	481	
0	0	0	74	143	197	233	297	333	356	416	464	
	0	1	2	3	4	5	6	7	8	9	10	i

Figura 7: Reconstrucción total del camino

Para más detalle del seguimiento se anexa en el repositorio un seguimiento visual del algoritmo.

4. Algoritmo para obtener el óptimo

A continuación presentamos el algoritmo de Programación Dinámica implementado para su posterior análisis de complejidad. Al momento de crear el algoritmo dividimos nuestro problema en dos partes: La construcción de la matriz de ganancias, y la búsqueda del máximo en la ultima columna. Esto lo hicimos de esta forma para luego poder utilizar la función que construye la matriz de ganancias y diseñar una función que reconstruya el plan de entrenamiento.

4.1. Algoritmo de programación dinámica

A continuación se presenta el algoritmo de programación dinámica diseñado para buscar la matriz de ganancias:

```
def obtenerGanancias(n, e_i, s_i):
  matriz = [ [0]*(n + 1) for _ in range(n + 1) ]
  matriz[0][0] = 0
  for i in range(1, n + 1):
    for j in range(0, i + 1):
      if j == 0:
        matriz[i][j] = max(matriz[i-1])
      else:
        matriz[i][j] = min(e_i[i - 1], s_i[j - 1]) + matriz[i - 1][j - 1]
  return matriz
```

Al observar el algoritmo implementado se puede observar que:

- Al crear la matriz se realizar n operaciones, resultando en una complejidad $O(n)$
- Al recorrer las columnas de la matriz e ir llenándolas con sus respectivos valores, se puede observar que en el peor de los casos (Ultima columna) se tarda n. Considerando esto para las n columnas, la complejidad es $O(n^2)$ (Debido a que solo se rellenan los valores de cada columna hasta el valor i, solamente se esta completando la mitad de la matriz, resultando en una complejidad $O(\frac{1}{2}n^2)$. No obstante, por notación O, se desprecian las constantes)

La ecuación de complejidad resultante es:

$$T(n) = O(n + n^2)$$

Simplificando, quedándonos solamente con la peor complejidad:

$$T(n) = O(n^2) \tag{1}$$

4.2. Algoritmo de ganancia máxima

```
def obtenerGananciaMaxima(n, e_i, s_i):
  return max(obtenerGanancias(n, e_i, s_i)[n])
```

Como se puede observar este algoritmo contiene la complejidad del algoritmo de programación dinámica, más la complejidad de buscar el máximo en una lista, resultando en la siguiente complejidad:

$$T(n) = O(n^2 + n) = O(n^2) \tag{2}$$

Pues nos quedamos con el peor de los casos.

4.3. Algoritmo de reconstrucción de entrenamientos

A continuación se presenta el algoritmo de programación dinámica diseñado para obtener el cronograma de entrenamientos:

```
def obtenerEntrenamientos(n, e_i, s_i):
  entrenamientos = []
  matriz = obtenerGanancias(n, e_i, s_i)
  posicionActual = matriz[n].index(max(matriz[n]))
  for i in range(n - 1, -1, -1):
    if posicionActual == 0:
      entrenamientos.insert(0, "Descanso")
      posicionActual = matriz[i].index(max(matriz[i]))
    else:
      entrenamientos.insert(0, "Entreno")
      posicionActual -= 1
  return entrenamientos
```

Como se puede observar este algoritmo contiene la complejidad del algoritmo de programación dinámica, más recorrer de manera lineal las listas. La ecuación de complejidad resultante es:

$$T(n) = O(n + n^2)$$

Simplificando, quedándonos solamente con la peor complejidad:

$$T(n) = O(n^2) \tag{3}$$

Pues nos quedamos con el peor de los casos.

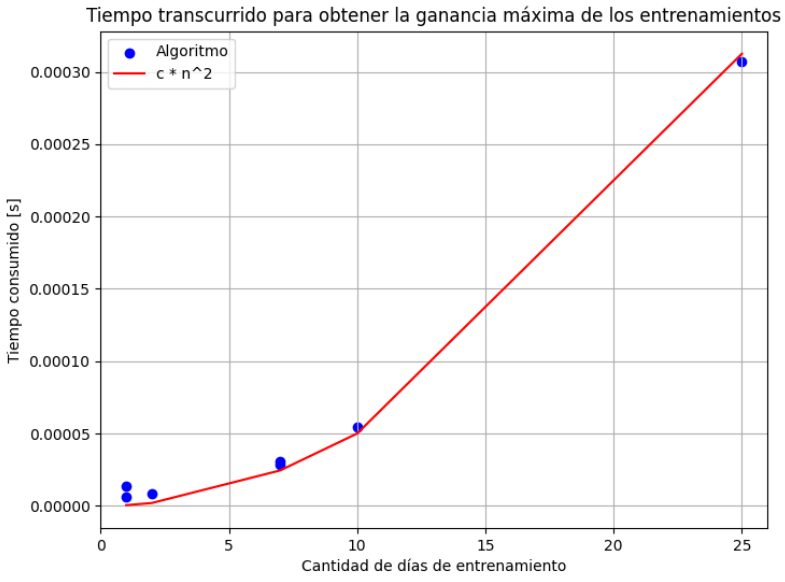
4.4. Análisis de optimalidad y tiempos

A continuación se realizaron una serie de pruebas para verificar que nuestro algoritmo obtiene los resultados óptimos para distintos sets de datos. Una vez que el algoritmo funcionaba correctamente, graficamos los tiempos de ejecución del algoritmo en función de la cantidad de días de entrenamiento.

Realizando un análisis de la variabilidad de los valores e_i y s_i notamos que ésta no afecta el tiempo en el que nuestro algoritmo encuentra la solución ni la optimalidad del mismo. Esto se debe a que la cantidad de subproblemas a resolver no se ve afectada por la variabilidad. No obstante, el volumen de entrenamientos a programar, como veremos en el análisis de complejidad temporal, aumenta los tiempos de ejecución del algoritmo, debido a que se aumenta la cantidad de operaciones a realizar.

Además, notamos que existen casos donde se pueden obtener dos o más secuencias óptimas que lleven a la ganancia máxima. Esto sucede principalmente en ejercicios con valores de esfuerzo y de energía parecidos.

Por otro lado podemos observar que se respeta la complejidad propuesta en el problema siendo cuadrática, patrón que se repetirá al hacer el mismo análisis con un volumen de datos notablemente mayor.

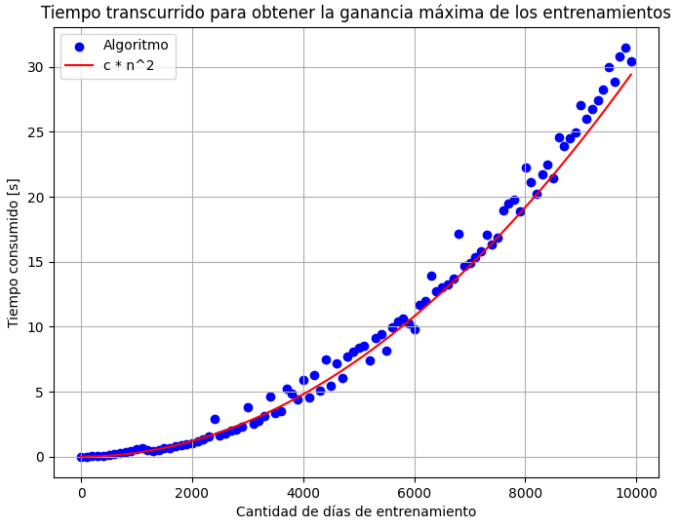


5. Mediciones

Se realizaron mediciones en base a pares de arreglos de diferentes largos, yendo de 100 en 100 elementos, donde los elementos en cada caso fueron generados por los valores pseudoaleatorios del lenguaje (el módulo `random`).

Para comprobar que la complejidad de nuestro algoritmo se encuentra correctamente calculada, realizamos mediciones del tiempo que el algoritmo desarrollado tardaba en encontrar la ganancia máxima para diferentes cantidades de elementos en pares de arreglos.

Para realizar estas mediciones, se realizó un bucle, en el cual se generaba el par de arreglos de esfuerzos y ganancias, contemplando que $s_i \geq s_{i+1}$, de n elementos aleatorios (Utilizando el paquete `random` de python), para los cuales se buscaba la ganancia máxima de dichos entrenamientos. Para esta búsqueda, se calculaba el tiempo que el algoritmo tardaba en llegar a la solución, y se lo guardaba en un arreglo. Este proceso se repitió para todas las iteraciones del bucle, incrementando la cantidad de elementos por arreglo, comenzando por 0 elementos, y aumentando hasta un máximo de 10000 elementos. Luego, una vez que se poseía el arreglo con todos los tiempos, para los 100 arreglos diferentes, se procedió a graficar los tiempos tardados, contra la cantidad de elementos en los arreglos. Sobre esto, se superpuso la gráfica de la función $f(n) = c \cdot n^2$ [1], para poder ver la similitud en tendencia del algoritmo, con la expresión de la complejidad calculada.



En el gráfico se puede observar como la nube de puntos tiende a seguir la curva establecida por la función $f(n) = c \cdot n^2$, indicándonos que el algoritmo posee una tendencia cuadrática. En consecuencia, esto nos asegura que la complejidad de nuestro algoritmo fue correctamente calculada.

[1] La constante c se utiliza para poder ajustar el conjunto de llegada de la función, para que sea en magnitud, similar a los valores obtenidos al calcular los tiempos del algoritmo. A un nivel físico, se la puede tomar como el producto del valor que representa el tiempo que tarda (en segundos) cada una de las operaciones de la computadora. Este valor fue obtenido de forma experimental.

6. Conclusiones

En primer lugar, bajo ciertas condiciones, un enfoque por programación dinámica permite construir soluciones óptimas de problemas más grandes a partir de soluciones óptimas a subproblemas más pequeños. Asimismo, la programación dinámica aplica el concepto de memoización, el cual nos permite guardar valores parciales pasados, evitándonos tener que recalcularlos. Estas características en conjunto nos permiten explorar potenciales espacios exponenciales de soluciones, de una forma más rápida y acotada.

En nuestro caso, hemos podido identificar y analizar el problema en cuestión, comparándolo con una variación del conocido problema "Juan el vago". Esta semejanza nos ha brindado la oportunidad de aplicar una descomposición de subproblemas, añadiendo una segunda dimensión que considera la cantidad de días desde el último descanso. De esta forma pudimos construir la ecuación de recurrencia que nos permite encontrar las solución óptima al problema.

Finalmente, logramos comparar la complejidad temporal calculada con mediciones de tiempo de nuestro algoritmo, logrando observar una buena aproximación a la calculada.